

Trabalho de Laboratório — Padrão Strategy em Java

Sistema de Robôs Autônomos

Objetivo

Implementar, em Java, um pequeno sistema de robôs usando o padrão **Strategy**. O objetivo é mostrar que certos comportamentos de um objeto podem variar independentemente da classe principal, podendo ser trocados em tempo de execução.

Contexto

Considere um sistema com diferentes tipos de robôs:

- RoboExplorador
- RoboSeguranca
- RoboEntrega
- RoboMedico
-

Todos herdam de uma classe abstrata chamada Robo.

Cada robô possui três comportamentos variáveis:

- locomoção
- comunicação
- ação principal
-

Esses comportamentos devem ser implementados como estratégias.

Estrutura esperada

Classe abstrata

- Robo

Subclasses

- RoboExplorador
- RoboSeguranca
- RoboEntrega
- RoboMedico

Interfaces de comportamento

- Locomocao
- Comunicacao
- Acao

Estratégias concretas

Locomoção

- MoverComRodas
- MoverComEsteiras
- Voar
-

Comunicação

- ComunicarPorRadio
- ComunicarPorWifi
-

Ação

- ExplorarAmbiente
- VigiarArea
- EntregarPacote
- PrestarSocorro

Parte 1 — Implementação

Implemente as interfaces:

```
interface Locomocao {  
    void mover();  
}
```

```
interface Comunicacao {  
    void comunicar();  
}
```

```
interface Acao {  
    void executar();  
}
```

Depois, implemente as classes concretas correspondentes.

Cada classe concreta deve imprimir uma mensagem simples, por exemplo:

Movendo com rodas

Comunicando por rádio

Explorando ambiente

Parte 2 — Classe abstrata

Robo

A classe Robo deve possuir os atributos:

```
protected Locomocao locomocao;  
protected Comunicacao comunicacao;  
protected Acao acao;
```

E os métodos:

```
public void mover()  
public void comunicar()  
public void executarAcao()
```

Esses métodos devem apenas **delegar** o comportamento para os objetos estratégia.

Também devem existir os métodos:

```
setLocomocao(...)  
setComunicacao(...)  
setAcao(...)
```

Parte 3 — Subclasses

Cada subclasse deve inicializar seus comportamentos no construtor.

Sugestão:

RoboExplorador → esteiras, rádio, explorar ambiente

RoboSeguranca → rodas, rádio, vigiar área

RoboEntrega → rodas, wifi, entregar pacote

RoboMedico → rodas, wifi, prestar socorro

Parte 4 — Teste no laboratório

Crie uma classe TesteRobos.

Ela deve:

1. Criar um vetor de robôs.
2. Percorrer o vetor.
3. Mostrar o nome da classe do robô.
4. Executar:
 - mover()
 - comunicar()
 - executarAcao()

Depois, escolha um robô e troque dinamicamente um comportamento usando um método set.

Exemplo:

```
robo.setLocomocao(new Voar());
```

Mostre novamente o comportamento alterado.

Entrega

O aluno deverá entregar apenas:

1. Diagrama de classes detalhado em UML.
2. Respostas às perguntas finais.
3. A saída do programa.

A implementação será verificada durante o laboratório.

O diagrama UML deve mostrar

- Herança entre Robo e suas subclasses.
- Interfaces Locomocao, Comunicacao e Acao.
- Classes concretas que implementam cada interface.
- Associação entre Robo e as três interfaces.
- Métodos principais de cada classe/interface.

Perguntas finais

1. Onde aparece a herança no projeto?
 2. Onde aparece a implementação de interfaces?
 3. Por que Robo possui atributos do tipo interface?
 4. O que significa dizer que Robo delega comportamento?
 5. Qual comportamento foi trocado dinamicamente no teste?
 6. Por que esse projeto é um exemplo do padrão Strategy?
 7. Que problema surgiria se todos os comportamentos fossem implementados com if/else dentro de Robo?
-

Restrições

- Não usar if ou switch dentro da classe Robo.
- Os comportamentos devem estar nas classes estratégia.
- O código deve compilar e executar corretamente.
- Usar nomes coerentes e boa indentação.